AI Financial Risk Intelligence Platform

A Technical Project Paper on Machine Learning, Explainable Risk Signals, Document Retrieval, and Analyst Decision Support

Author: Abhishek Upadhyay
Repository: github.com/Abhishek4209/AI-Financial-Risk-Intelligence-Platform-using-Graph-RAG-Multi-Agent-LLM
Reviewed branch: lfs-fix-from-remote
Paper type: Source-grounded portfolio project paper

Abstract

Financial risk workflows often require more than a single prediction: analysts need a risk signal, an interpretable view of influential features, access to relevant policy or regulatory material, and a way to organize review cases. This project presents a prototype financial-risk intelligence platform that combines dataset-specific credit-default and transaction-fraud models with SHAP-based feature attribution, document ingestion and retrieval-augmented question answering, and selected case/customer operations. A React/Vite interface communicates with a FastAPI backend; PostgreSQL supports operational records, while FAISS is the default local vector-search backend. The repository also contains Neo4j graph-query utilities and additional retrieval strategies. A source audit distinguishes active API behavior from optional or standalone utilities, avoiding the assumption that all modules form one end-to-end Graph-RAG or multi-agent pipeline. The platform is intended for analyst decision support and experimentation, not autonomous financial decisions.

Keywords: Financial risk, credit default, fraud detection, XGBoost, Logistic Regression, SHAP, RAG, embeddings, FAISS, BM25, reranking, Neo4j, FastAPI, explainable AI

1. Introduction

Financial services teams examine heterogeneous signals: application attributes, transaction behavior, supporting documents, and regulatory guidance. A prediction-only interface can provide a score but may not make the basis for review clear or help an analyst find source material. The platform aims to bring several review-oriented capabilities into one prototype: credit and fraud scoring, feature-driver explanations, document Q&A;, investigation summaries, and case/customer views.

> The core design principle is human review. Model outputs are estimated risk signals. They do not establish that a transaction is fraudulent, a customer will default, or a legal obligation has been met. Retrieved passages and generated summaries require source verification.

2. System Architecture and Implementation

The application has a React/Vite frontend and a FastAPI service layer. The frontend exposes risk, fraud, compliance, document, and operational views. API schemas use Pydantic validation. PostgreSQL and SQLAlchemy support selected overview, customer, and case operations. The default local vector-store integration uses FAISS; Chroma, Milvus, and Pinecone adapters are present as configurable alternatives. Neo4j supports relationship-query utilities, but graph evidence is not consistently joined to the main compliance RAG answer context.

The repository contains Docker and Compose configuration plus GitHub Actions CI checks for Python compilation, tests/coverage, frontend installation/build, and Docker image build. These are CI/build checks—not proof of a production deployment. The reviewed configuration does not demonstrate a completed cloud deployment workflow.

3. Machine-Learning Models and Prediction

Credit-risk scoring uses scikit-learn Logistic Regression pipelines for Taiwan credit-card default and Lending Club loan-status risk. The documented pipeline applies numeric imputation and standardization and uses class weighting. The model returns the class-1 score through predict_proba. The API maps scores into review bands: HIGH at or above 0.50, MEDIUM from 0.25 to below 0.50, and LOW below 0.25. These bands are application thresholds, not calibrated certainty.

Fraud scoring includes two XGBoost model paths. The PaySim model is trained with 500 estimators, depth 6, learning rate 0.05, subsampling and column sampling of 0.8, and imbalance weighting derived from training data. Feature construction includes transaction time fields, balance-error magnitudes, zero-balance indicators, amount-to-origin-balance ratio, and one-hot transaction type. The IEEE-CIS model uses 400 estimators, depth 6, learning rate 0.05, subsampling/column sampling of 0.8, an AUC-PR evaluation metric, and class-ratio weighting. Its training pipeline merges transaction and identity data and handles missing numeric values using -999.

The project also includes a generic training utility for Logistic Regression or Random Forest with preprocessing and evaluation outputs. It is separate from the dataset-specific prediction artifacts. Model algorithm claims in this paper are based on training notebook source and matching inference paths; serialized artifacts were not deserialized during the audit.

4. Explainability

SHAP is used to expose positive and negative feature contributions for selected credit and fraud predictions. The implementation selects a TreeExplainer for tree models and attempts linear or generic explainers for other estimators. These explanations help an analyst understand which input features influenced a model output. SHAP values are feature attributions, not causal proof. One code path uses a zero-valued transformed background when no background data is provided, which should be considered when interpreting results.

5. Document Processing and Retrieval-Augmented Generation

The document workflow parses supported document formats, cleans text, splits it into chunks, attaches source metadata, embeds the chunks, and indexes them for retrieval. The default embedding model is BAAI/bge-small-en-v1.5 with normalized embeddings and CPU as the default execution device. API chunking defaults are 1,000 characters with 150 characters of overlap; other ingestion utilities may use their own settings. FAISS is the default local index. The active search path uses a top-k setting defaulting to five (API-constrained to 1–20) and returns source/page metadata where available.

The active Q&A; flow expands a query through deterministic synonym/risk-term substitutions, performs vector retrieval for up to four variants, removes exact duplicate chunks, applies a heuristic relevance filter, and reranks candidates with the cross-encoder cross-encoder/ms-marco-MiniLM-L-6-v2. If reranking fails, keyword overlap is used as a fallback. The generation step formats selected source passages into a grounded-answer prompt and includes source references.

A critical implementation distinction: BM25 lexical retrieval and weighted hybrid BM25/vector fusion are implemented as utilities, but the main active Q&A; service does not call them. The same applies to LLM query rewriting and parent-child retrieval. The active Self-RAG-like relevance check is deterministic lexical/category logic, not an autonomous reflection loop or semantic entailment guarantee.

6. Graph Analytics and Investigation

Neo4j utilities model relationships among customers, accounts, transactions, and documents and support queries such as connected-customer patterns, fan-in/fan-out behavior, transaction chains, and customer-document associations. Selected fraud investigation logic can attach graph evidence. However, graph building and querying are not universally wired into the primary compliance RAG route, and document ingestion does not currently populate the Neo4j document graph. Therefore, the system should be described as having graph analytics capabilities rather than as a fully integrated Graph-RAG system.

The investigation service combines a transaction score, selected SHAP drivers, retrieved regulatory or financial evidence, and a generated analyst-oriented summary. This path is distinct from the direct fraud-model scoring endpoint and uses its own retrieval/model configuration.

7. Reported Evaluation and Interpretation

Saved credit-model comparison reports contain the following results. They are reproduced as stored experiment values and are not a fresh run or an independent validation of production performance.

Dataset	Test rows	ROC-AUC	PR-AUC	Precision	Recall	F1
Taiwan Credit Card	6,000	0.7081	0.4904	0.3672	0.6202	0.4613
Lending Club	273,516	0.6608	0.3342	0.2977	0.6356	0.4055

A saved IEEE-CIS threshold comparison reports threshold 0.44 with precision 0.2082, recall 0.7547, and manual-review rate 0.1247; threshold 0.51 with precision 0.2506, recall 0.7032, and manual-review rate 0.0966. These are stored experiment rows, not the current active inference threshold. The PaySim notebook records evaluation metrics, but exact values were not verified in the source audit.

8. Security, Testing, and Deployment Considerations

The repository contains authentication middleware and JWT-related configuration, but the frontend request helper does not attach a bearer token; protected browser calls therefore need an external token mechanism or development-mode authentication bypass. The Compose configuration provisions PostgreSQL and the backend, while readiness checks also expect Redis and Neo4j. This service mismatch should be resolved before treating readiness as a reliable deployment signal. Unit tests and CI definitions are present, but no test run or passing coverage result is asserted in this paper.

For deployment hardening, the project needs a coherent authentication flow, service-aligned readiness checks, secrets management, database migrations, robust evidence-grounding evaluation, fairness and temporal validation, artifact provenance, monitoring, and deployment manifests. Model artifacts and datasets should be versioned and used only under their licenses and terms.

9. Limitations and Responsible Use

This work is a portfolio/educational prototype. Risk scores are not proof of fraud or default; SHAP is not causal explanation; generated answers may be incorrect; and a HIGH band should trigger review rather than automatic blocking, loan rejection, or regulatory filing. The active system does not fully integrate every advertised module. In particular, the graph and BM25/hybrid capabilities should not be presented as the main live RAG path without further integration and testing.

10. Conclusion

The AI Financial Risk Intelligence Platform demonstrates a multi-component approach to financial analyst support: supervised credit/fraud scoring, feature attribution, source-aware document retrieval, generated investigation summaries, and operational case interfaces. Its strongest contribution is the combination of several review tools in one prototype while retaining human oversight. Future work should focus on integrating graph evidence into retrieval, validating all inference and document paths end to end, improving model and RAG evaluation, and completing secure deployment and governance.

References and Source Basis

This paper is based on the repository source audit and project overview supplied for this conversation: source modules for credit/fraud inference, SHAP explainability, document ingestion, embedding/retrieval, graph query utilities, API schemas/routes, frontend API calls, notebooks, saved metric reports, Docker/Compose configuration, and GitHub Actions workflow. The paper intentionally distinguishes code-level capabilities from active runtime wiring.

Repository:
https://github.com/Abhishek4209/AI-Financial-Risk-Intelligence-Platform-using-Graph-RAG-Multi-Agent-LLM/tree/lfs-fix-from-remote

Audit note: This is a technical project paper, not a peer-reviewed publication. Evaluation figures are from saved reports and should be independently reproduced before use in a formal research claim.